Update 15: Architecture Comparison—Autoregressive, Diffusion, and State Space Models

KV Cache Strategies, Time Complexity, and Inference Benchmarking with vLLM

Nathan Gopee
MS Computer Science Candidate
SUNY New Paltz
gopeen1@newpaltz.edu

February 2026

Abstract

This update compares three fundamentally different language model architectures—autoregressive Transformers (GPT-oss via vLLM [8, 11]), discrete diffusion models (MDLM [12], SEDD [10]), and selective state space models (Mamba/Granite 4 [4, 6])—with emphasis on how each handles inference state (KV cache vs. recurrent state vs. iterative denoising), time complexity scaling, and practical throughput on consumer GPU hardware. We derive the per-token and total generation cost for each paradigm, compare memory scaling with sequence length, and present a benchmarking plan for the lab cluster (RTX 3090, RTX 5090, Tesla V100).

1 Three Paradigms for Language Model Inference

Modern language models fall into three architectural families, each with distinct inference characteristics:

1. **Autoregressive Transformers**: Generate tokens left-to-right, one per forward pass. Each step attends to all prior tokens via a KV cache that grows linearly with context. Examples: GPT-4, Llama, GPT-oss [11].
2. **Diffusion Language Models**: Start with a fully masked or noised sequence and iteratively denoise all tokens in parallel over T refinement steps ($T \ll n$). No KV cache; each step recomputes full bidirectional attention. Examples: MDLM [12], SEDD [10], Mercury [7].
3. **State Space Models (SSMs)**: Process tokens sequentially through a fixed-size recurrent state that compresses all history into $O(d \cdot N)$ values per layer, independent of sequence length. No KV cache. Examples: Mamba [4], Mamba-2 [3], Granite 4 [6].

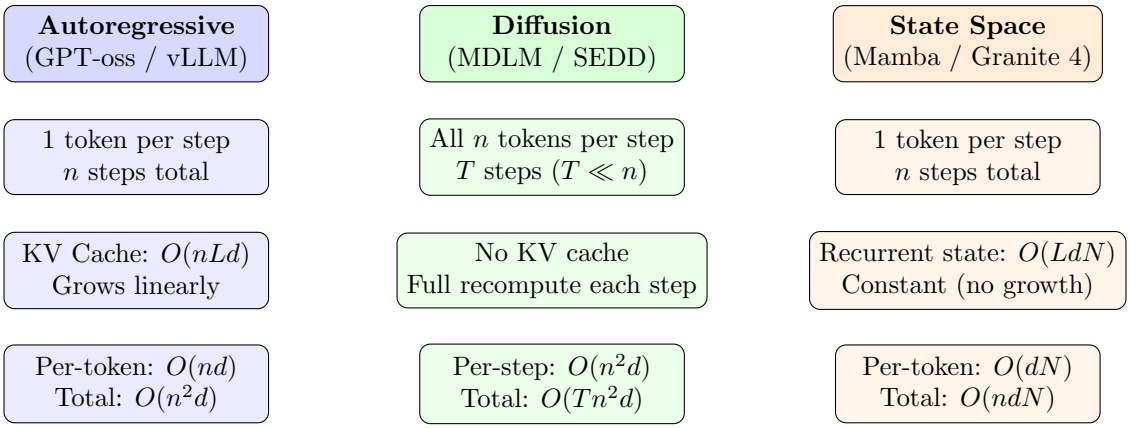

Figure 1: Comparison of inference characteristics across three architecture paradigms. n=sequence length, d=model dimension, L=layers, N=SSM state dimension (typically 16), T=diffusion steps (typically 32–256).

2 Autoregressive Transformers: GPT-oss via vLLM

2.1 GPT-oss Architecture

GPT-oss-120b [11] is an open-source Mixture-of-Experts (MoE) model with 116.8 billion total parameters. Using MXFP4 quantization, the model compresses to approximately 63 GB, enabling deployment across consumer GPUs via tensor parallelism in vLLM [8]. The MoE architecture routes each token to a subset of expert FFN blocks (top-2 out of 16 experts), so the active parameter count per token is substantially lower than the total.

2.2 KV Cache in Autoregressive Models

The KV cache stores the key and value projections from all prior tokens at every attention layer, enabling each new token to attend to the full context without recomputing all past representations.

For a model with L layers, H_{kv} KV heads (under Grouped Query Attention), and head dimension d_k:

$$\text{KV cache size} = 2 \cdot L \cdot H_{kv} \cdot d_k \cdot n \cdot b \tag{1}$$

where n is the current sequence length and b is the bytes per element (2 for FP16). For a 7B-class model ($L = 32$, $H_{kv} = 8$, $d_k = 128$):

The per-token KV cache addition is constant (128 KB), but the attention computation grows linearly because each new token must attend to all cached keys. At 128K context, the KV cache alone consumes 16 GB—a significant fraction of an RTX 3090's 24 GB VRAM.

Table 1: KV cache memory scaling for a 7B-class Transformer

Context Length	KV Cache	Per-Token Cost	Attention FLOPs/token
512	64 MB	128 KB	6.7×10^7
2,048	256 MB	128 KB	2.7×10^8
8,192	1 GB	128 KB	1.1×10^9
32,768	4 GB	128 KB	4.3×10^9
131,072	16 GB	128 KB	1.7×10^{10}

2.3 vLLM PagedAttention

vLLM [8] addresses KV cache fragmentation via PagedAttention, which stores KV blocks in non-contiguous GPU memory pages (analogous to virtual memory). This eliminates internal fragmentation and enables dynamic batching of requests with different context lengths. For the RDMA middleware, PagedAttention's block-granular KV management maps naturally to RDMA scatter/gather operations for disaggregated prefill-decode serving.

2.4 Time Complexity

- **Prefill** (processing prompt of length p): $O(L \cdot p^2 \cdot d)$ for self-attention across the prompt. Parallelizable across the p tokens.
- **Per-token generation**: $O(L \cdot (n \cdot d + d^2))$ where n is the current context. The $n \cdot d$ term (attention against KV cache) dominates at long contexts. The d^2 term (FFN) is constant.
- **Total generation** of n tokens: $O(L \cdot n^2 \cdot d)$ because attention cost sums as $1 + 2 + \cdots + n = n(n+1)/2$.

Practical throughput. On a single RTX 3090 (24 GB), a 7B model in FP16 generates ∼30–50 tokens/sec at short context (<4K) but degrades to ∼10–15 tokens/sec at 128K context as the KV cache fills VRAM and attention becomes memory-bandwidth bound. vLLM's continuous batching mitigates this for concurrent requests.

3 Diffusion Language Models: MDLM and SEDD

3.1 Discrete Diffusion for Text

Unlike image diffusion (which operates in continuous pixel/latent space), text diffusion must handle a discrete vocabulary. Two dominant approaches have emerged:

- **Absorbing-state / Masked diffusion**: Tokens are progressively replaced with a `[MASK]` token during the forward process. The reverse process learns to unmask them. This connects directly to masked language modeling (BERT). Used by MDLM [12] and D3PM [2] (absorbing variant).

- **Score-based discrete diffusion**: Rather than predicting masked tokens, the model learns probability ratios ("concrete scores") for transitioning between discrete states. Used by SEDD [10].

3.2 MDLM: Masked Diffusion Language Models

MDLM [12] (Sahoo et al., NeurIPS 2024) simplifies discrete diffusion by casting it as time-weighted masked language modeling. The forward process masks tokens according to a schedule $\alpha(t)$, and the training loss reduces to:

$$\mathcal{L} = \mathbb{E}_{t,x_0}\left[\frac{-\alpha'(t)}{\alpha(t)} \sum_{i:x_t^i=\texttt{[M]}} \log p_\theta(x_0^i \mid x_t, t)\right] \tag{2}$$

where $\alpha(t) = (1-t)^{1/s}$ is a geometric noise schedule with learned exponent s. At GPT-2 scale ($\sim$110M parameters), MDLM achieves perplexity of $\sim$27.2 on OpenWebText, slightly outperforming GPT-2 small ($\sim$29.4 perplexity).

3.3 SEDD: Score Entropy Discrete Diffusion

SEDD [10] (Lou, Meng, Ermon; ICML 2024) parameterizes the "concrete score"—the ratio of transition probabilities in discrete diffusion:

$$s_\theta(x_t, t)[i, j] = \frac{p_\theta(x_0 = j \mid x_t, t)}{p_\theta(x_0 = x_t^i \mid x_t, t)} \tag{3}$$

This avoids computing intractable normalizing constants. SEDD achieves $\sim$31.4 perplexity on OpenWebText at GPT-2 scale and establishes the theoretical foundations that led to Mercury [7], the first commercially deployed diffusion LLM (developed by Inception Labs, co-founded by SEDD author Aaron Lou and Stefano Ermon).

3.4 No KV Cache—Full Recompute Each Step

Diffusion LLMs use **bidirectional attention** (no causal mask) because all tokens are visible at every denoising step. This means:

- No KV cache accumulation between generation steps—each of the T diffusion steps performs a complete forward pass over the full n-token sequence.
- Each step costs $O(L \cdot n^2 \cdot d)$ for full bidirectional self-attention.
- Total generation: $O(T \cdot L \cdot n^2 \cdot d)$ where T is the number of denoising steps.

The total FLOP count may exceed autoregressive generation, but the **wall-clock time** can be competitive because: (1) T is typically 32–256, much smaller than n (e.g., 1024 tokens); and (2) each step is fully parallelizable, saturating GPU compute.

3.5 Reducing Diffusion Steps

3.5.1 Self-Distillation Through Time (SDTT)

SDTT adapts consistency distillation [13] from continuous image diffusion to discrete text. The model serves as both teacher (EMA parameters θ^-) and student (current parameters θ), enforcing temporal consistency:

$$f_\theta(x_t, t) \approx f_{\theta^-}(\hat{x}_{t-\Delta t}, t - \Delta t) \tag{4}$$

Progressive halving ($T \to T/2 \to T/4 \to \cdots$) reduces 1000+ steps to 32–64 steps with <10% perplexity degradation, yielding ∼16–32× speedup.

3.5.2 Confidence-Based Unmasking

Rather than unmasking tokens in random or fixed order, confidence-based scheduling unmasks the highest-confidence predictions first:

$$c_i = \max_v p_\theta(x_0^i = v \mid x_t, t) \tag{5}$$

At each step, the top-k most confident positions are committed (where $k = n/T$). This improves generation quality by 10–20% perplexity over random-order unmasking, as confident tokens provide context for harder predictions.

3.5.3 Curriculum Learning

Training diffusion LLMs benefits from noise schedule curriculum: beginning with low masking ratios (10–30%, similar to BERT pre-training) and gradually increasing to 70–99%. This allows the model to learn local token relationships before global structure, stabilizing training and accelerating convergence.

3.6 Approximate KV Caching for Diffusion: dLLM-Cache and dKV-Cache

Although diffusion LLMs lack a traditional KV cache, recent work introduces caching between denoising steps:

- **dLLM-Cache**: Monitors per-token confidence across diffusion steps. Tokens whose predictions remain stable for K consecutive steps are "frozen" and their hidden-state representations are cached, avoiding redundant computation. Reports 1.5–3× speedup with negligible quality loss.
- **dKV-Cache**: Caches KV pairs from the previous diffusion step. Only recomputes KV for tokens that changed significantly (newly unmasked or revised), using approximate attention for frozen tokens. Reports 2–5× speedup in attention computation, with aggressive settings trading 3–5% quality for higher throughput.

3.7 Block Diffusion: Hybrid AR + Diffusion

Block Diffusion [1] (Arriola, Sahoo et al.) divides the output into blocks of B tokens. Blocks are generated autoregressively (left to right), but tokens within each block are generated via masked diffusion in parallel. This enables unbounded sequence length (unlike pure diffusion's fixed-length constraint) while retaining intra-block parallelism:

$$p(x_1, \ldots, x_n) = \prod_{b=1}^{n/B} p_{\text{diff}}(x_{bB+1:bB+B} \mid x_{1:bB}) \tag{6}$$

With $B = 128$ and $T = 32$ diffusion steps per block, generating 1024 tokens requires $8 \times 32 = 256$ total forward passes, each processing only $B = 128$ tokens. This yields 2–4× speedup over pure autoregressive while maintaining left-to-right coherence across blocks.

4 State Space Models: Mamba and Granite 4

4.1 Structured State Space Models

The Mamba architecture [4] (Gu & Dao, 2023) builds on Structured State Spaces (S4) [5]. A continuous-time state space model maps input $x(t)$ to output $y(t)$ through a latent state $h(t)$:

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \qquad y(t) = \mathbf{C}h(t) \tag{7}$$

where $\mathbf{A} \in \mathbb{R}^{N \times N}$ governs state dynamics, $\mathbf{B} \in \mathbb{R}^{N \times 1}$ projects input into state space, $\mathbf{C} \in \mathbb{R}^{1 \times N}$ projects state to output, and N is the state expansion factor (typically $N = 16$). After zero-order hold discretization with step size Δ:

$$h_t = \bar{\mathbf{A}}h_{t-1} + \bar{\mathbf{B}}x_t, \qquad y_t = \mathbf{C}h_t \tag{8}$$

where $\bar{\mathbf{A}} = \exp(\Delta \cdot \mathbf{A})$ and $\bar{\mathbf{B}}$ is derived from Δ and $\mathbf{B}$.

4.2 The Selective Mechanism

In S4, $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and Δ are fixed (input-independent), making the system Linear Time-Invariant (LTI). This enables efficient convolution-mode computation but prevents content-based reasoning. Mamba's key innovation is making $\mathbf{B}$, $\mathbf{C}$, and Δ **input-dependent**:

$$\mathbf{B}_t = W_B x_t, \quad \mathbf{C}_t = W_C x_t, \quad \Delta_t = \text{softplus}(W_\Delta x_t) \tag{9}$$

This makes the system time-varying, enabling selective memory: the model can choose what to store in and retrieve from the recurrent state based on input content. The cost is that convolution-mode computation is no longer possible, requiring a hardware-aware parallel scan algorithm.

4.3 Inference: Constant-Cost Token Generation

During inference, Mamba processes one token at a time via the recurrence in eq. (8). The per-token cost is:

$$\text{Per-token FLOPs} = O(L \cdot d \cdot N) \quad \text{where } N = 16 \tag{10}$$

This is **independent of sequence length**—the model's "memory" of prior tokens is compressed into the fixed-size recurrent state $h \in \mathbb{R}^{d \times N}$, which is updated in-place at each step. There is no KV cache, no growing attention matrix, and no degradation at long contexts.

Table 2: Inference state memory: Transformer KV cache vs. Mamba recurrent state (7B-class model)

Context Length	Transformer KV Cache	Mamba State	Ratio
512	64 MB	13 MB	5×
2,048	256 MB	13 MB	20×
8,192	1 GB	13 MB	79×
32,768	4 GB	13 MB	315×
131,072	16 GB	13 MB	1,260×

At 128K context, the Mamba recurrent state is ∼1,260× smaller than the Transformer KV cache. This has profound implications for RDMA-based distributed inference: transferring 13 MB of state between nodes costs ∼0.14 ms on 100 GbE ConnectX-4, while transferring 16 GB of KV cache costs ∼1,300 ms.

4.4 Mamba-2: Structured State Space Duality

Mamba-2 [3] (Dao & Gu, ICML 2024) establishes a mathematical duality between structured SSMs and linear attention, showing that a scalar-identity SSM is equivalent to causal linear attention with a specific decay structure. This enables a hybrid algorithm (SSD) that uses:

- **Within chunks** (Q tokens, e.g., 256): Quadratic "attention-like" computation via matrix multiply, fully exploiting GPU tensor cores.
- **Between chunks**: Recurrent state passing, compressing each chunk's information into the state.

This yields 2–8× speedup over Mamba-1's selective scan while maintaining comparable quality. Mamba-2 also introduces multi-head SSM (analogous to multi-head attention), improving expressiveness and tensor-parallelism compatibility.

4.5 Granite 4: Hybrid SSM-Attention

IBM's Granite 4 [6] adopts a hybrid architecture inspired by Jamba [9] (AI21 Labs), interleaving Mamba layers with sparse attention layers at approximately a 7:1 ratio. This preserves the memory

efficiency of SSMs (only attention layers maintain KV caches) while retaining the precise recall capability of attention for tasks like needle-in-a-haystack retrieval.

Jamba demonstrated this approach at scale: with 52B total parameters (12B active via MoE), Jamba's KV cache at 256K context is ∼4 GB—compared to ∼194 GB for Llama 2 70B at the same context—because only ∼4 out of 32+ layers use attention. Granite 4 applies this hybrid design to enterprise-focused models with transparent training data provenance.

5 Formal Complexity Comparison

Let n denote output sequence length, d model dimension, L number of layers, N SSM state dimension, and T diffusion steps.

Table 3: Asymptotic complexity comparison across three paradigms

Operation	**Transformer (AR)**	**Diffusion**	**Mamba (SSM)**
Prefill (p tokens)	$O(Lp^2d)$	N/A	$O(LpdN)$
Per-token generation	$O(L(nd + d^2))$	N/A	$O(L(dN + d^2))$
Per-step cost	—	$O(Ln^2d)$	—
Total gen. (n tokens)	$O(Ln^2d)$	$O(TLn^2d)$	$O(LndN)$
State memory	$O(Lnd)$	$O(Lnd)$ (temp.)	$O(LdN)$
State growth	Linear in n	Fixed per step	Constant

Key observations.

1. The Transformer's total generation cost is $O(Ln^2d)$ because attention cost accumulates as $\sum_{i=1}^{n} i = O(n^2)$. Mamba achieves $O(LndN)$—linear in n—because each token costs $O(dN)$ regardless of position.
2. Diffusion's total cost is $O(TLn^2d)$, which has a higher FLOP constant than autoregressive. However, each of the T steps is fully parallelizable across all n tokens, and $T \ll n$ (e.g., $T = 64$, $n = 1024$). When $T < n$, the wall-clock time (assuming sufficient parallelism) is $O(T \cdot n^2)$ vs. $O(n^2)$ for AR—competitive when T is small.
3. For the RDMA middleware, the state transfer cost is critical. The Transformer KV cache ($O(Lnd)$ bytes) dominates transfer time for disaggregated prefill-decode. Mamba's constant-size state ($O(LdN)$) makes state migration between nodes nearly free. Diffusion models transfer no persistent state between steps.

6 Practical Throughput and Memory Analysis

6.1 Estimated Throughput by Architecture

The critical observation is that Mamba maintains **constant throughput** regardless of context length, while Transformer throughput degrades as the KV cache grows. Diffusion models have high

Table 4: Estimated generation throughput on a single RTX 3090 (24 GB, FP16)

Model	**Architecture**	**Params**	**tok/s (4K ctx)**	**tok/s (128K ctx)**
GPT-oss-120b (TP=4)	AR Transformer	116.8B	∼15	∼3
Llama-class 7B	AR Transformer	7B	∼45	∼12
MDLM (T=64)	Diffusion	110M	∼1,000	∼200
MDLM (T=256)	Diffusion	110M	∼300	∼60
Mamba-2.8B	Pure SSM	2.8B	∼100	∼100
Jamba (12B active)	Hybrid SSM+Attn	52B	∼60	∼50

throughput at small scale (110M) but the per-step cost of full bidirectional attention over the entire sequence limits scaling to longer outputs.

6.2 Memory Budget at Inference

Table 5: VRAM budget breakdown at 32K context, FP16, single GPU

Component	**7B AR**	**110M Diffusion**	**2.8B Mamba**	**Jamba 52B (INT4)**
Model weights	14 GB	0.2 GB	5.6 GB	13 GB
KV cache / state	4 GB	0 GB	0.013 GB	0.5 GB
Activations	1 GB	0.5 GB	0.5 GB	1 GB
Total	**19 GB**	**0.7 GB**	**6.1 GB**	**14.5 GB**
Fits RTX 3090?	Yes (tight)	Yes	Yes	Yes (INT4)

Mamba's 13 MB recurrent state is negligible compared to model weights. For a 7B Transformer at 128K context, the KV cache (16 GB) exceeds the model weights (14 GB), making it the dominant memory consumer. This asymmetry motivates KV-cache-aware RDMA scheduling in the middleware.

7 Testing Plan: Three-Way Architecture Benchmark

7.1 GPT-oss via vLLM (Autoregressive Baseline)

The existing vLLM deployment for GPT-oss-120b uses the `vllm/vllm-openai:gptoss` Docker image with MXFP4 quantization and FlashInfer MoE optimization. The benchmark measures:

- **Time-to-first-token (TTFT)**: Prefill latency for prompt lengths 128, 512, 2048, 8192.
- **Tokens/second**: Generation throughput at context lengths from 1K to 32K.
- **KV cache memory**: Measured via `nvidia-smi` at each context length.
- **RDMA vs. TCP**: Compare throughput when tensor-parallel shards communicate over ConnectX-4 RDMA vs. TCP (using the existing benchmark script).

7.2 MDLM / SEDD (Diffusion)

Both MDLM and SEDD have open-source implementations with pretrained checkpoints at GPT-2 scale (∼110M parameters):

- **MDLM**: Clone `kuleshov-group/mdlm`, download the OpenWebText checkpoint. Evaluate perplexity and measure generation time at varying diffusion steps ($T = 32, 64, 128, 256, 1024$).
- **SEDD**: Clone `louaaron/Score-Entropy-Discrete-Diffusion`, evaluate with the same step sweep.
- **Step-quality tradeoff**: Plot perplexity vs. wall-clock time as T varies. Identify the "knee" where step reduction yields diminishing quality loss.
- **Multi-GPU**: Since diffusion steps are embarrassingly parallel across the sequence, test whether data parallelism across V100 × 2 (Tower 2) or RTX 3090 × 3 (Chimera) yields proportional speedup.

7.3 Mamba / Granite 4 (State Space)

- **Mamba-2.8B**: Download from Hugging Face (`state-spaces/mamba-2.8b-slimpj`). Measure generation throughput at context lengths from 1K to 128K, verifying that per-token latency remains constant.
- **Granite 4**: Test IBM's hybrid SSM-Attention model via `ibm-granite` checkpoints. Compare KV cache size (attention layers only) against a pure Transformer of similar parameter count.
- **vLLM serving**: Deploy Mamba-2.8B through vLLM's `MambaForCausalLM` backend. Compare serving throughput against vLLM serving Llama-7B at matched context lengths.
- **State migration**: Measure the time to serialize and transfer Mamba's recurrent state (∼13 MB) vs. a Transformer's KV cache (∼4 GB at 32K) over RDMA, quantifying the migration advantage for elastic serving.

7.4 Benchmark Matrix

Table 6: Three-way benchmark matrix

Metric	**GPT-oss (AR)**	**MDLM (Diff.)**	**Mamba (SSM)**	**Unit**
Generation throughput	vLLM serve	PyTorch script	vLLM serve	tok/s
TTFT	vLLM API	N/A	vLLM API	ms
Perplexity (OWT)	N/A	Eval script	Eval script	ppl
VRAM at 32K ctx	`nvidia-smi`	`nvidia-smi`	`nvidia-smi`	GB
State size at 32K	KV cache dump	N/A	State dump	MB
State transfer (RDMA)	`ib_write_bw`	N/A	`ib_write_bw`	ms
Quality (text gen)	Human eval	Human eval	Human eval	subjective

8 Relevance to RDMA Middleware

The three architectures interact differently with the RDMA middleware:

- **AR Transformers**: KV cache is the dominant data to transfer for disaggregated inference. The middleware's hot/cold tensor classification determines whether KV blocks are cached locally (hot) or fetched on-demand via RDMA (cold). PagedAttention's block-granular KV layout maps to RDMA scatter/gather.
- **Diffusion LLMs**: No persistent state to transfer between nodes. Instead, the middleware could distribute diffusion steps across GPUs—each GPU denoises a subset of the sequence (spatial parallelism) or a subset of the diffusion steps (temporal parallelism). The RDMA cost is exchanging intermediate token predictions between steps, which is $O(n \cdot V)$ per synchronization point.
- **Mamba/SSM**: The ∼13 MB recurrent state makes live migration nearly free over RDMA (<1 ms on 100 GbE). This enables aggressive elastic scheduling: tasks can be migrated between GPUs mid-generation with negligible overhead, unlike Transformers where KV cache transfer dominates migration cost.

9 Conclusions

1. Autoregressive Transformers (GPT-oss via vLLM) provide the highest quality and most mature serving infrastructure, but their $O(Lnd)$ KV cache grows linearly with context, creating a memory bottleneck at long sequences and a transfer bottleneck for RDMA-based disaggregated serving.
2. Diffusion LLMs (MDLM, SEDD) eliminate the KV cache entirely and generate all tokens in parallel, but each of the T denoising steps requires full $O(n^2)$ bidirectional attention. Step reduction via SDTT and confidence-based scheduling reduces T to 32–64 steps, and approximate caching (dLLM-Cache, dKV-Cache) adds 2–5× speedup within each step. Block Diffusion hybridizes AR and diffusion for unbounded sequence length with intra-block parallelism.
3. Mamba/SSM models (Granite 4) compress all context into a fixed-size recurrent state ($O(LdN)$, ∼13 MB for 2.8B parameters), achieving constant per-token cost and constant memory regardless of sequence length. Hybrid SSM-Attention architectures (Jamba, Granite 4) retain precise recall via sparse attention layers while keeping KV caches ∼8× smaller than pure Transformers.
4. For the RDMA middleware, each architecture presents a different optimization target: KV cache partitioning and on-demand fetch for Transformers, step-parallel distribution for diffusion models, and near-zero-cost state migration for SSMs. The benchmark plan will quantify these tradeoffs on the lab cluster.

References

[1] Marianne Arriola, Subham Sekhar Sahoo, Aaron Gokaslan, Yair Schiff, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *arXiv preprint*, 2024.

[2] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. *NeurIPS*, 2021.

[3] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *ICML*, 2024.

[4] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[5] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *ICLR*, 2022.

[6] IBM Research. IBM Granite language models. https://github.com/ibm-granite, 2025.

[7] Inception Labs. Mercury: Diffusion-based language model. https://inceptionlabs.ai/, 2025.

[8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[9] Opher Lieber, Barak Lenz, Horace Bata, et al. Jamba: A hybrid Transformer-Mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

[10] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *ICML*, 2024.

[11] OpenAI. GPT-oss-120b: Open-source mixture-of-experts language model. https://huggingface.co/openai/gpt-oss-120b, 2025.

[12] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *NeurIPS*, 2024.

[13] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. *ICML*, 2023.